

Understanding and Mitigating Hallucinations in Industrial LLM-based Unit Test Generation

Yanlun Tu^{*}
Ant Group
Shanghai, China
tuyanlun.tyl@antgroup.com

Ziyue Zhou^{*}
University of Electronic Science and
Technology of China
Chengdu, China
zhouziyue0309@gmail.com

Cheng Xu[†]
Ant Group
Shanghai, China
kunqiu.xc@antgroup.com

Jingling Sun[†]
University of Electronic Science and
Technology of China
Chengdu, China
jingling.sun910@gmail.com

Shuai Feng
Ant Group
Shanghai, China
fengshuai.fs@antgroup.com

Chengyu Zhang
Loughborough University
Loughborough, United Kingdom
c.zhang4@lboro.ac.uk

Abstract

Unit testing plays a critical role in ensuring software quality and reliability in large-scale industrial environments. While Large Language Models (LLMs) offer promising automated test generation capabilities, their practical deployment faces significant challenges due to hallucination problems. In this paper, we analyze compilation failures from LLM-generated unit tests in Ant Group's production systems and identify two fundamental types of hallucinations: extrinsic hallucinations caused by insufficient contextual information and intrinsic hallucinations stemming from model limitations even with adequate context. To address these issues, we propose DEHALL, an automated end-to-end unit test generation tool that systematically mitigates both types of hallucinations through comprehensive context construction and targeted static analysis-based repair. Our approach builds a heterogeneous graph to capture relevant context and employs specialized repair mechanisms for import, field, and method issues. Evaluation on Ant Group's internal datasets reveals that DEHALL achieves 71.56% line coverage and 67.18% branch coverage, significantly outperforming vanilla LLM approaches. In the public benchmarks, it also shows better performance on coverage and better defect detection capability than previous state-of-the-art approaches. DEHALL has been successfully deployed across multiple business domains at Ant Group, achieving an 81% developer adoption rate with positive user feedback on productivity improvements.

Keywords

unit test, large language models, static analysis

ACM Reference Format:

Yanlun Tu, Ziyue Zhou, Cheng Xu, Jingling Sun, Shuai Feng, and Chengyu Zhang. 2026. Understanding and Mitigating Hallucinations in Industrial

LLM-based Unit Test Generation. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26)*, July 5–9, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3803437.3805260>

1 Introduction

Unit testing serves as a foundation of software quality assurance, particularly in large-scale industrial environments where system reliability directly impacts business and user trust. At Ant Group, where our financial systems serve hundreds of millions of users, comprehensive unit testing is a critical requirement for maintaining system stability and regulatory compliance. However, the scale and complexity of our codebase, accumulated over years of rapid development and spanning thousands of microservices, present significant challenges for traditional unit testing approaches.

The traditional approach to unit testing depends largely on manual test case development by software engineers. It creates significant bottlenecks in modern development workflows. Developers spend a large portion of their time writing unit tests, especially for complex business code with intricate logic.

The automated unit test generation techniques aim to solve this productivity bottleneck. In the past decades, the techniques have significantly progressed, from random-based generation algorithms [4, 6, 17] that produce syntactically valid but semantically trivial tests, to search-based algorithms like EvoSuite [8] that systematically explore the test space via sophisticated evolutionary algorithms. While these methods can generate valid test cases, they often produce tests with low coverage, poor readability, and low diversity, making them impractical for industrial adoption.

The recent rise in the code generation capabilities of Large Language Models (LLMs) [14] inspires researchers to explore their application in unit test generation. Our initial attempt, which we term the Vanilla LLM approach, allows developers to select code snippets and automatically generate corresponding unit tests using state-of-the-art LLMs. However, our practical experience reveals that one of the major limitations preventing the deployment of Vanilla LLMs is that the generated test cases frequently suffer from compilation failures. Developers spend significant time manually correcting these LLM-generated unit tests, even finding it more efficient to write tests from scratch rather than debugging them.

^{*}Equal Contributions.

[†]Corresponding authors: Cheng Xu and Jingling Sun.



This work is licensed under a Creative Commons Attribution 4.0 International License.
FSE Companion '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2636-1/26/07
<https://doi.org/10.1145/3803437.3805260>

To this end, we collected LLM-generated unit tests that have compilation failures from Ant Group’s internal development process and systematically categorized the logs of compilation failures. Our analysis shows that these compilation failures can be attributed to two primary LLM hallucinations: extrinsic hallucinations, due to insufficient context, and intrinsic hallucinations, due to the LLM’s inherent limitations. This observation motivated us to develop DEHALL, an automated end-to-end unit test generation tool that systematically addresses the above hallucination problems.

To mitigate extrinsic hallucinations, DEHALL constructs a comprehensive context by extracting the classes, methods, and fields relevant to the focal method for test generation. To mitigate intrinsic hallucinations, DEHALL automatically resolves compilation failures via specialized test case repair methods that target issues related to imports, fields, and methods.

To evaluate the effectiveness of DEHALL in real-world industrial development environments, we first conducted experiments on Ant Group’s internal industrial dataset. The dataset comprises 162 normal-complexity focal methods sampled from 32 production applications and 29 high-complexity focal methods selected from 18 mission-critical applications, reflecting the diverse testing demands across Ant Group’s business systems. Compared with the Vanilla LLM approach, DEHALL achieves substantial gains on both datasets, with improvements of 45.22% and 29.21% in line and branch coverage on the normal-complexity dataset, and 34.78% and 51.61% on the high-complexity dataset.

In addition, we compared DEHALL with state-of-the-art approaches. Given that many existing approaches cannot be reliably adopted in our internal production environments, we further evaluated DEHALL on the widely used Defects4J benchmark [16]. The results show that DEHALL significantly outperforms both traditional search-based methods (EvoSuite [8]) and recent LLM-based test generation approaches (ChatUniTest [3]) across representative projects. Specifically, DEHALL achieves 79.54% line coverage and 73.96% branch coverage, outperforming EvoSuite (67.01% / 69.70%) and ChatUniTest (65.07% / 60.04%), while also requiring less execution time (38.89s vs. 60s and 77.40s, respectively).

Overall, these evaluations demonstrate that DEHALL delivers strong effectiveness in industrial settings while maintaining state-of-the-art performance on public academic benchmarks. At present, DEHALL has been successfully deployed across multiple business units within Ant Group’s development environment, which is used by thousands of developers, with a developer adoption rate exceeding 81%. The feedback from the developers highlights notable productivity gains and reduces the manual testing overhead.

In summary, this paper has the following contributions:

- We collect hallucinations in LLM-based unit test generation in our real-world industrial environments and conduct an empirical study to understand their manifestations and types.
- We present DEHALL, an automated unit test generation tool that eliminates hallucinations through comprehensive context capturing and specialized test case repairing.
- We demonstrate the effectiveness of DEHALL through extensive evaluation on both an industrial dataset and a public benchmark, showing significant improvements over existing approaches in real-world settings.

Table 1: Breakdown of failures in the Vanilla LLM approach.

Failure Category	Percentage	Hallucination Type
Undefined Symbols	49.03%	Extrinsic
Type Errors	18.55%	Intrinsic/Extrinsic
Syntax Errors	13.71%	Intrinsic
Dependency Errors	9.84%	Intrinsic/Extrinsic
Access Permission Errors	6.29%	Intrinsic
Ambiguous Method Call	2.58%	Intrinsic

2 Motivation

At Ant Group, where complex financial systems built by Java serve hundreds of millions of users, the need for comprehensive unit testing for Java Programs is particularly critical. In our development environment, engineers use integrated development environments (IDEs) to write business code for various financial applications. While writing business logic is their primary responsibility, ensuring the correctness of each functional module through appropriate testing is equally important. This creates a demand for rapid and flexible test case generation that can keep pace with agile development cycles. To support this need, we integrated large language model (LLM) capabilities into our development tools, allowing developers to generate unit tests for selected code snippets simply by prompting the model. The prompt includes the source code of the target method along with instructions specifying the expected test format and coding standards. We refer to this basic LLM-based test generation method as the Vanilla LLM approach.

However, our experience with the Vanilla LLM approach has encountered significant challenges in our industry scenario. To understand the challenges, we instrumented the test cases generated by our user-developers and collected 620 compilation results of the test cases for one month. We then investigated and categorized the logs of compilation failures and summarized the manifestations and hallucination types. The results are shown in Table 1.

2.1 Manifestations Analysis

As shown in Table 1, we summarize the manifestations of the compilation failures into six categories with percentages:

- (1) *Undefined Symbols* are the most prevalent failures, accounting for 49.03% of all failures. The manifestations of this kind of failure include generating tests that reference classes, methods, or variables that do not exist in the codebase.
- (2) *Type Errors* refer to incorrect method parameter types or return assignments that contradict the defined signatures.
- (3) *Syntax Errors* are violations of grammar rules, such as missing semicolons, mismatched brackets, incorrect keyword usage, or malformed statements, which prevent the code from being successfully compiled or parsed.
- (4) *Dependency Errors* refer to incorrect usage of testing frameworks or mocking libraries, such as attempting to mock final classes or using outdated API patterns.
- (5) *Access Permission Errors* refer to the direct invocation of private methods without using reflection or appropriate accessor patterns, violating the encapsulation principles.

```
// File: AccountType.java (Dependency in a different package)
public enum AccountType {
    SAVINGS,
    CHECKING
    // Note: "ENTERPRISE" does not exist in the actual codebase
}
// File: AccountService.java
public class AccountService {
    public boolean processAccount(AccountType type) {
        // Logic handling account types
    }
}
// --- Vanilla LLM Generated Test (Extrinsic Hallucination) ---
public class AccountServiceTest {
    @Test
    public void testProcessAccount() {
        // Arrange
        // The LLM, lacking the context of the AccountType definition,
        // hallucinates a plausible enum value "ENTERPRISE".
        // Compilation Failure here
        AccountType expectedType = AccountType.ENTERPRISE;
        AccountService service = new AccountService();
        // Act
        boolean result = service.processAccount(expectedType);
        // Assert
        assertTrue(result);
    }
}
```

Figure 1: Example of Extrinsic Hallucination: Vanilla LLM generates a non-compilable test by hallucinating an undefined enum value ENTERPRISE.

- (6) *Ambiguous Method Calls* are scenarios where the generated test invokes a method without providing sufficient type information to resolve the target method, typically due to method overloading or incompatible implicit conversions.

2.2 Hallucination Taxonomy

By analyzing these failures in sequence, we observe that they are primarily caused by LLM hallucinations [13]. As shown in Table 1, we further categorize these hallucinations into two main types: Extrinsic Hallucinations and Intrinsic Hallucinations.

Extrinsic Hallucinations are the hallucinations caused by a lack of sufficient contextual information regarding the focal method, such as the correct definition of related classes, methods, and fields. These hallucinations are not due to the limited capabilities of LLMs but to the insufficient information provided to them. For example, in Figure 1, consider a method `processAccount` that accepts an `AccountType` enum parameter defined in a separate package. The Vanilla LLM approach, lacking access to the definition of the external dependency `AccountType`, hallucinates a plausible but non-existent enum value `ENTERPRISE`. This generates a test that references an undefined symbol, rendering the code non-compilable.

Intrinsic Hallucinations are the hallucinations caused by the limited capabilities of LLMs, even when LLMs have sufficient contextual information. This type of hallucination in test generation is unavoidable because LLMs intrinsically have hallucinations. For instance, consider the `transferFunds` method that enforces strict type safety for financial data to ensure precision as Figure 2 shows. While the Vanilla LLM correctly identifies the business intent of the method, it fails to obey the strict type signature required for financial operations. By hallucinating a string input (e.g., `"100.50"`) instead of the precise `BigDecimal` object, the generated test code

```
// File: AccountService.java
import java.math.BigDecimal;
public class AccountService {
    // Transfers a specific amount.
    // Throws a checked exception InsufficientFundsException.
    public void transferFunds(String accountId, BigDecimal amount)
        throws InsufficientFundsException {
        // ... transfer logic
    }
}
// --- Vanilla LLM Generated Test (Intrinsic Hallucination) ---
public class AccountServiceTest {
    @Test
    public void testTransferFunds() {
        AccountService service = new AccountService();
        String id = "ACC-001";
        try {
            // [Type Error]: The LLM hallucinates the parameter type,
            // passing a String literal "100.50" instead of the required
            // BigDecimal object. This contradicts the method signature.
            // Compilation Failure here
            service.transferFunds(id, "100.50");
        } catch (InsufficientFundsException e) {
            fail("Transfer failed unexpectedly");
        }
    }
}
```

Figure 2: Example of Intrinsic Hallucination (Type Error): Vanilla LLM generates a test that passes a String literal instead of required BigDecimal, causing a compilation failure.

fails to compile and ignores the critical context of floating-point arithmetic precision inherent in financial systems. This illustrates a specific type of Intrinsic Hallucination where the model possesses the correct semantic knowledge of the method yet applies syntactically incorrect types.

These hallucination patterns are not merely academic concerns; they represent real barriers to adoption in production environments. Our internal metrics and survey indicate that developers spend significant time manually correcting these generated tests, often finding it faster to write tests from scratch rather than debugging LLM-generated code. The severity of these issues motivated us to develop a more sophisticated approach that addresses both information deficiency and logical inconsistency problems.

3 Method

To address the hallucination problems identified in our motivation study, we propose DEHALL, an automated end-to-end unit test generation tool. While this tool was developed for Java to meet Ant Group’s specific business requirements, the underlying methodologies are language-agnostic and applicable to other programming languages.

Figure 3 illustrates our complete workflow, which consists of three phases. When a developer selects a focal method for unit test generation, our tool first constructs a comprehensive context graph that captures all relevant classes, methods, and fields. This context graph is then provided to the LLM for initial unit test generation. Finally, a test case repair phase systematically identifies and repairs any remaining potential compilation failures before presenting the generated tests to the developer.

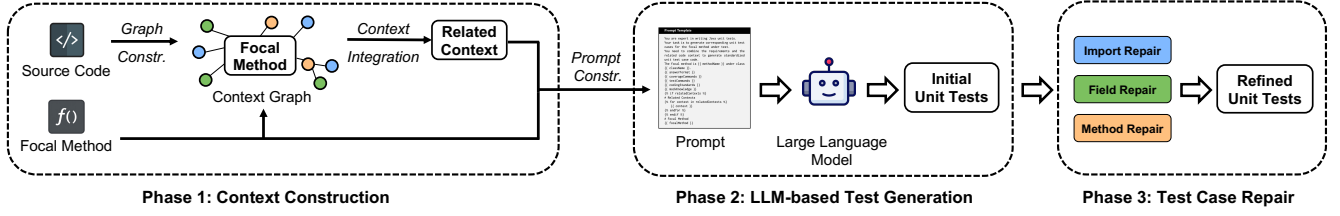


Figure 3: Overview of DEHALL's three-phase workflow.

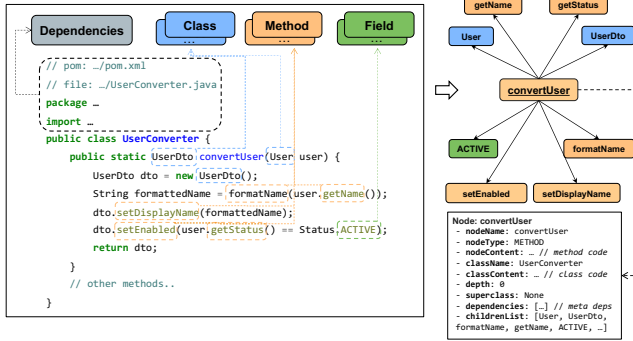


Figure 4: Heterogeneous graph construction process showing the transformation from source code to a dependency graph.

3.1 Phase 1: Context Construction

The methods under test in the industry codebases at Ant Group heavily rely on external dependencies, such as methods and fields in the same or other classes. The Vanilla LLM approach, which focuses solely on the focal method, leads to Extrinsic Hallucinations due to missing contextual information. To mitigate these hallucinations, in this phase, DEHALL constructs a context graph from the focal method to extract key dependencies. It then ranks these dependencies to generate an optimized context, enabling LLMs to produce higher-quality unit tests in the next phase.

3.1.1 Context Graph Construction. The context graph construction step begins with building a comprehensive heterogeneous graph that maintains the dependencies of the focal method. Starting from the focal method as the root node, we extract all classes, methods, and fields referenced by the focal method, and then recursively perform the same analysis on each identified method.

Algorithm 1 shows the algorithm for constructing the context graph. The algorithm constructs a heterogeneous context graph $G = (V, E)$ rooted at a focal method m_f . Employing a depth-limited recursive search up to D_{max} , the procedure iteratively identifies program elements via dependency analysis. For each valid method node v , dependencies u are added to the vertex set V if not already present. Edges are simultaneously established in E , explicitly classifying the relationship between v and u as either a superclass extension or a child dependency.

Figure 4 illustrates an example of our context graph construction process. Starting from a focal method named `convertUser`, our

Algorithm 1: Context Graph Construction

Input: Focal method m_f , Maximum search depth D_{max}

Output: Heterogeneous graph $G = (V, E)$

Let v_{root} be the node corresponding to m_f ;

Set $Depth(v_{root}) \leftarrow 0$;

$V \leftarrow \{v_{root}\}$, $E \leftarrow \emptyset$;

Function $BUILDGRAPH(v, d)$

if $d \geq D_{max}$ **then**

return;

 Let S be the set of program elements derived from $DEPENDENCYANALYSIS(v)$;

if $Type(v) \neq METHOD$ **then**

return;

foreach $u \in S$ **do**

if $u \notin V$ **then**

$V \leftarrow V \cup \{u\}$;

$BUILDGRAPH(u, d + 1)$;

 Let $e = (v, u)$;

if u is super class of v **then**

$Super(v) \leftarrow u$;

else

$Children(v) \leftarrow Children(v) \cup \{u\}$;

$E \leftarrow E \cup \{e\}$;

$BUILDGRAPH(v_{root}, 0)$;

return $G(V, E)$;

graph construction identifies downstream related dependencies, including classes `User` and `UserDto`, methods `formatName`, `getName`, `setEnabled`, and `getStatus`, along with the field `ACTIVE`. Each of these dependencies becomes a node in the heterogeneous context graph, with edges representing their dependency relationships to the focal method.

3.1.2 Context Prioritization. Complex focal methods often involve extensive dependencies, which can cause the required context to exceed the context window of LLMs. To address this challenge, we implement a prioritization mechanism to incorporate as many relevant dependencies as possible within the limited context constraint.

To reflect the developer's focus on inputs, outputs, and internal method calls during unit testing, we assign the highest priority to local instance methods and classes of parameters and return type

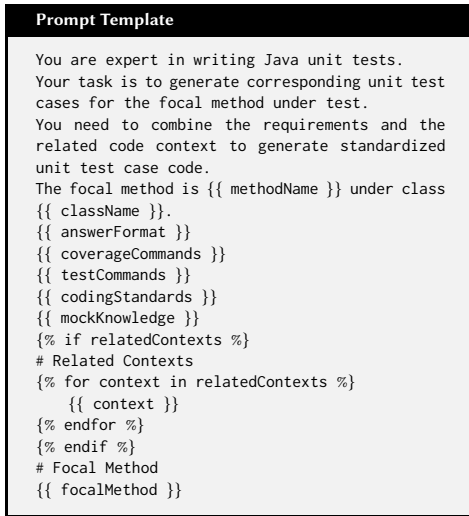


Figure 5: Example of Prompt Template used in DEHALL by Jinja2 [27] syntax.

for the focal method. Local instance methods are prioritized to ensure a deep understanding of the focal logic. In traditional practice, by analyzing the local instance method, the developer can discover more branching logic and thereby generate tests with higher coverage. Classes of parameters and return type are prioritized to ensure the construction of valid test cases. This is because parameters dictate the requirements for mocking and variable initialization, while return type defines the necessary logic for assertion and verification.

The remaining dependencies are sorted first by depth in the context graph and then by content length, favoring lower depth and shorter length. Finally, they are ordered by dependency type, where enumerable classes take the highest priority, followed by methods, fields, and other classes.

At last, to respect the fixed context window constraints, we define a MAX_TOKEN threshold. We select dependencies in the ranked order and truncate the sequence once the cumulative length of the included nodes exceeds this limit.

3.1.3 Context Integration. In the final phase of context construction, we consolidate nodes belonging to the same class in the context graph into a unified class-level definition. This ensures that the context presented to the LLM has complete and syntactically valid class structures. For example, the methods `setDisplayname` and `setEnabled` belong to class `UserDto`. We retain these method definitions within the `UserDto` class while filtering out irrelevant definitions. Consequently, these elements are merged into a single node with nodeContent coherent class definition. The final remaining nodes after context integration would be prompted to LLMs as related contexts.

3.2 Phase 2: LLM-based Test Generation

With enriched context from heterogeneous graph construction, we prompt LLM to generate initial unit tests. As illustrated in Figure 5

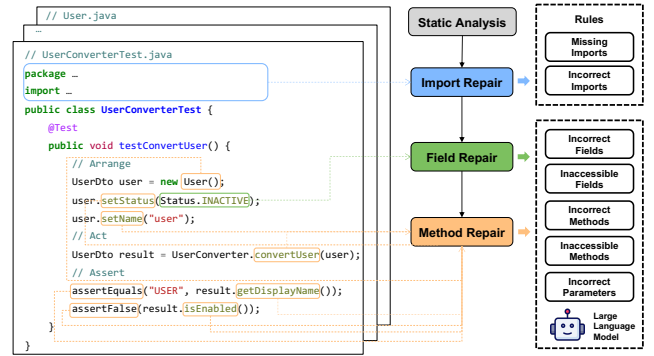


Figure 6: Overview of the test case repair process for resolving intrinsic hallucinations in generated unit tests.

with Jinja2 syntax [27], our prompt design is structured into several key components to guide the generation process effectively:

Role Definition and Task Description: The prompt begins by establishing the LLM as an expert in Java unit testing. It clearly defines the objective: to generate standardized test cases for the specific focal method identified by its class and method name.

Generation Guidelines: To ensure the quality and standardization of the output, we incorporate specific constraints including the *answer format*, specific *coverage commands*, specific *testing commands*, and adherence to *coding standards*. *mock knowledge* provides the guides on how to handle external test dependencies correctly.

Related Contexts: This section provides the retrieved and consolidated code snippets from the previous phases. By supplying the necessary class structures and dependencies directly within the prompt, we mitigate common issues by extrinsic hallucinations.

Focal Method: The code of the focal method under test is presented at the end of the prompt. This serves with high attention for the LLM to analyze the logic and branch conditions, enabling the generation of test cases that accurately target the method's behavior.

With this structured prompt, the LLM generates initial test cases that utilize the context provided by Phase 1.

3.3 Phase 3: Test Case Repair

Despite comprehensive context provision, LLM-generated tests may still contain Intrinsic Hallucinations like logical inconsistencies that violate established facts or constraints. Our test case repair phase first identifies issues through static analysis and then resolves them through rules and LLMs. Test Case Repair adopts static analysis over direct compilation to enable fast, non-intrusive, and preemptive repair of test cases. This server-side strategy delivers repaired test cases before reaching the user's IDE environment, avoiding the disruption and latency of local compilation, and overcoming the limited actionable guidance that compiler errors provide for repair.

Figure 6 illustrates our test case repair process, which operates as a systematic refinement pipeline. The process begins with static analysis of the LLM-generated test cases to detect three categories of issues: import errors, field errors, and method errors. These issues are then addressed by our specialized strategy: the Import Repair handles missing or incorrect import statements, the Field Repair

resolves field access and type mismatches, and the Method Repair corrects accessibility, type mismatch, and method signature issues. The import errors are fixed by specific rules, and the latter two errors are fixed by LLMs. After applying the repair, the system re-parses the refined unit tests by static analysis to validate corrections and identify any remaining problems.

Building upon the heterogeneous graph from the context construction phase, we perform a light-weight static analysis on the generated tests to identify hallucinations. The key is to map the class, method, and field usage within the generated test code against the concrete class definitions stored in the graph.

3.3.1 Import Issue Identification and Repair. Import issues are characterized by missing or incorrect import statements that prevent compilation. Import Repair component, which is powered by rules, focuses on resolving compilation dependencies caused by missing or incorrect import statements. It scans the generated test for class references within the heterogeneous graph to identify required imports. For other testing framework dependencies such as JUnit [15], TestNG [30], Mockito [22], or PowerMock [26] not explicitly detailed in the graph, we maintain a curated mapping of these libraries to determine correct imports based on the project’s Maven POM files. Additionally, to correct hallucinated imports, the component validates each import statement against actual class definitions in the graph, replacing mismatched entries with correct alternatives.

3.3.2 Field Issue Identification and Repair. Field issues involve references to non-existent fields or violations of access control. Field Repair component targets field-related errors by first scanning and validating each reference against the complete class hierarchy. It traces the origin of field accesses based on inheritance relationships to verify both field existence and type compatibility. Upon detecting specific issues such as type mismatches, incorrect fields, or inaccessible private and protected members, the component constructs a detailed prompt with the relevant code context and the specific error analysis. This prompt is then fed into the LLM to generate precise fixes, which include applying appropriate conversion methods, alternative fields, or generating reflection-based access code with proper error handling.

3.3.3 Method Issue Identification and Repair. Method issues encompass incorrect method signatures, inaccessible methods, or parameter mismatches. Method Repair component addresses method-related hallucinations by first performing a rigorous scan to validate method signatures against actual definitions, including return types and accessibility. This inspection also traces back to base classes to ensure the coverage of the class hierarchy. Upon detecting issues such as incorrect methods, parameter mismatches, or inaccessible private and protected members, the component provides the LLM with the specific error details and relevant code context. The LLM then generates precise fixes, such as changing to other closely matching methods or compatible overloaded methods, and generating reflection-based invocation code.

4 Evaluation

We conduct a comprehensive evaluation of DEHALL to answer four key research questions:

RQ1: How effective and efficient is DEHALL in generating unit tests in real-world industrial scenarios?

RQ2: What is the contribution of each key module in DEHALL to the overall test generation performance?

RQ3: What is the impact of different LLMs on the performance of our proposed method?

RQ4: How does DEHALL compare with state-of-the-art techniques on public benchmarks?

4.1 Experimental Setup

This section outlines the experimental configurations applicable to all research questions. To evaluate the performance of generated unit test cases, **Line Coverage** and **Branch Coverage** are adopted as core metrics. They assess the number of lines and branches in the focal method covered by unit test cases, respectively. To quantify the efficiency of the generation process, we also measure the average end-to-end **Time** of generation for each focal method. All experiments run on a server with Ubuntu 20.04 and an Intel Xeon Gold 6430 (3.40 GHz) processor.

4.1.1 Experiment Setup of RQ1. To evaluate DEHALL’s performance in real industrial settings, we constructed two benchmark datasets using proprietary production code repository from Ant Group, grouped by the complexity of the focal methods:

Normal Complexity: Comprising 162 focal methods from 32 production applications across diverse business domains, representing common development scenarios for standard unit testing.

High Complexity: A curated set of 29 focal methods from 18 core applications, selected in collaboration with senior developers based on Halstead complexity metrics [11], cyclomatic complexity metric [7], the number of cross-module dependencies, and business criticality. These methods represent the most challenging edge cases for unit test generation.

4.1.2 Experiment Setup of RQ2. To quantify the contribution of each DEHALL component, we conduct systematic ablation experiments across both datasets. We evaluate three configurations:

Baseline: Vanilla LLM approach without context construction or static analysis repair.

+Context: Baseline enhanced with our heterogeneous graph-based context construction.

+Context+Repair: Complete DEHALL implementation with both context construction and test case repair.

To intuitively assess the usability of the generated test cases, we introduce **Compilation Success Rate**, which measures whether the test cases generated for each focal method can be compiled and executed directly. Furthermore, to quantify the efficiency and scalability of the generation process, we add the average output **Tokens** of LLMs for each focal method.

4.1.3 Experiment Setup of RQ3. To understand how LLMs affect DEHALL’s performance, we replace the underlying LLMs used in the LLM-based Test Generation and Test Case Repair phases with five representative models: DeepSeek-V3 [18], DeepSeek-V3.1 [18], DeepSeek-V3.2 [19], Kimi-K2-Instruct [29], and Qwen3-Coder [32]. The evaluation metrics are the same as those used in RQ2.

Table 2: Ablation Study Results on Two Benchmark Datasets.

Dataset	Configuration	Compilation Success Rate	Line Coverage	Branch Coverage	Time (s)	Tokens
Normal Complexity Dataset	Baseline	23.46%	26.34%	38.97%	37.18	953.24
	+Context	42.59%	55.73%	63.59%	35.76	992.37
	+Context+Repair	64.81%	71.56%	67.18%	40.76	1104.73
High Complexity Dataset	Baseline	3.45%	-	-	78.96	1748.55
	+Context	20.69%	5.46%	9.68%	75.82	1760.86
	+Context+Repair	58.62%	34.78%	51.61%	89.61	1938.93

Table 3: Distribution of Issues Addressed by Test Case Repair.

Repair Component	Number of Issues	Percentage
Import Repair	40	53.33%
Field Repair	12	16.00%
Method Repair	23	30.67%

4.1.4 Experiment Setup of RQ4. To evaluate DEHALL against traditional search-based and recent LLM-based test generation approaches, we select two representative baselines for comparison:

EvoSuite [8]: A widely used search-based approach. We adopt official default configuration for EvoSuite with latest 1.1.0 version.

ChatUniTest [3]: A recent LLM-based approach. For fair comparison, we configure both ChatUniTest and DEHALL to use the same underlying LLM (DeepSeek-V3).

We compare DEHALL with these methods on the public benchmark Defects4J [16]. It provides a well-established benchmark for Java unit testing research. We adopt the latest v3 version for experiments. This dataset contains 17 open-source projects with 854 real faults, from which we extract 200 related focal methods for evaluation. Beyond basic evaluation metrics, we also evaluate the bug detection capability with the case study.

4.2 Experimental Results

4.2.1 Results for RQ1. DEHALL with default LLM of DeepSeek-V3 achieves superior performance on both normal and high complexity datasets. On the normal-complexity dataset, DEHALL attains 71.56% line coverage and 67.18% branch coverage with compile success rate of 64.81%. The average cost time for the focal methods is 35.76s. As for the result on the high-complexity dataset, DEHALL still maintains 34.78% line coverage and 51.61% branch coverage with a compile success rate of 58.62%. The average cost time is raised to 89.61s for this harder dataset. Based on the experimental results, we observe that for focal methods in industrial settings, which are typically characterized by complex dependencies and branching logic, DEHALL is able to generate complete and usable unit test cases. The generated test cases maintain rich semantic information, cover diverse boundary scenarios, and achieve high coverage rates.

To quantify practical utility of DEHALL in real-world industrial development, we measure *adoption rate*, defined as the proportion of generated code that developers commit to the code base at the line level. The online results show that DEHALL improves the adoption rate from 57% to 81%, demonstrating a significant improvement in developer acceptance. This improvement can be attributed to

several key factors: our context construction provides comprehensive semantic information that aligns with development practices, the test case repair mechanism ensures compilation success by addressing common syntactic and dependency issues, and the tailored prompt instructions satisfy internal business testing requirements.

Despite achieving an adoption rate of 81%, some generated code remains uncommitted. Through analysis of rejected code lines, we identify three primary reasons for non-adoption:

- (1) *User Removal.* Developers may manually remove test cases they consider unnecessary or redundant based on their specific understanding of the codebase context.
- (2) *Mock Data Modification.* Developers often adjust mock data content to better align with actual production scenarios or internal test data requirements, resulting in code changes that are not directly adopted.
- (3) *Code Style Adaptation.* Developers may modify generated code to conform to specific repository coding standards, naming conventions, or architectural patterns, leading to customizations that deviate from original generated code.

4.2.2 Results for RQ2. To evaluate the contribution of individual components in DEHALL, we conduct ablation studies on both the normal and high complexity datasets. We analyze the impact of *Context Construction* and *Test Case Repair* on compilation success rate and coverage, with the detailed results summarized in Table 2.

Performance on Normal Complexity Dataset. As shown in the first three rows of Table 2, adding context construction significantly enhances performance over the baseline. Specifically, it improves the compilation success rate by 19.13% (from 23.46% to 42.59%), line coverage by 29.39% (from 26.34% to 55.73%), and branch coverage by 24.62% (from 38.97% to 63.59%). Building upon context construction, test case repair provides further substantial gains, increasing the compilation success rate by an additional 22.22% to 64.81%, line coverage by 15.83% to 71.56%, and branch coverage by 3.59% to 67.18%. These results demonstrate that both components are essential for achieving optimal test generation quality.

Performance on High Complexity Dataset. The results on the high-complexity dataset, presented in the last three rows of Table 2, highlight even more pronounced improvements. The baseline method struggles severely, with only one focal method test passing compilation and zero code coverage (marked as -). Context construction improves the compilation success rate by 17.24% (to 20.69%) and achieves line coverage of 5.46% and branch coverage of 9.68%. With the addition of test case repair, our full approach achieves a 37.93% improvement in compilation success rate (reaching 58.62%),

Table 4: LLM Performance Comparison on Two benchmark Datasets.

Dataset	Model	Compilation Success Rate	Line Coverage	Branch Coverage	Time (s)	Tokens
Normal Complexity Dataset	DeepSeek-V3	64.81%	71.56%	67.18%	35.76	1104.73
	DeepSeek-V3.1	57.41%	61.38%	62.05%	40.01	1676.10
	DeepSeek-V3.2	46.91%	61.15%	58.20%	70.73	2780.24
	Qwen3-Coder	53.09%	48.35%	45.58%	38.69	1487.32
	Kimi-K2-Instruct	61.11%	68.11%	70.71%	43.07	1679.75
High Complexity Dataset	DeepSeek-V3	58.62%	34.78%	51.61%	89.61	1938.93
	DeepSeek-V3.1	44.83%	35.87%	30.56%	138.96	3020.34
	DeepSeek-V3.2	37.93%	29.09%	33.33%	150.46	5522.28
	Qwen3-Coder	44.83%	38.64%	30.00%	115.93	3238.79
	Kimi-K2-Instruct	41.38%	18.42%	21.42%	142.99	3999.86

Table 5: Performance Comparison on Defects4J Dataset.

Method	Line Coverage	Branch Coverage	Time (s)
EvoSuite	67.01%	69.70%	60s
ChatUniTest	65.07%	60.04%	77.40s
DEHALL	79.54%	73.96%	38.89s

a 29.32% improvement in line coverage (reaching 34.78%), and a 41.93% improvement in branch coverage (reaching 51.61%). This progression underscores that our approach is critical for handling complex industrial scenarios where the baseline fails.

Analysis of Efficiency and Token Usage. Correspondingly, we observe a gradual increase in average cost time and output token length as components are added. However, an interesting phenomenon emerges in the context construction ablation: the average time actually *decreases* compared to the baseline (e.g., from 37.18s to 35.76s on the normal-complexity dataset). We attribute this counterintuitive efficiency gain to the reduction of hallucinations. In the baseline configuration, the lack of sufficient context causes the model to generate hallucinated repetitive contents (e.g., infinite loops of code) in certain cases. These repetitive generation tasks significantly inflate the execution time. Therefore, providing comprehensive context not only improves the correctness and coverage of the generated test cases but also prevents hallucination-driven overhead, thereby enhancing overall responsiveness.

Contribution Analysis of Test Case Repair Components. To further understand the effectiveness of different repair strategies within the test case repair phase, we analyze the distribution of issues identified and resolved by the three repair components. Table 3 presents the detailed breakdown of issues addressed by each repair component. The statistics show that among the 75 compilation issues detected, Import Repair addresses 40 issues (53.33%); Field Repair resolves 12 issues (16.00%); and Method Repair handles 23 issues (30.67%). This distribution reveals that import-related hallucinations are the most prevalent issue, likely due to missing or incorrect dependencies in complex industrial codebases. Field and method issues indicate significant limitations in LLMs’ understanding of class hierarchies or the usage of fields and methods, reinforcing the necessity of comprehensive context construction combined with targeted repair mechanisms.

4.2.3 Results for RQ3. Table 4 presents the performance of evaluated LLMs on normal and high complexity datasets, respectively.

Performance on Normal Complexity Dataset. As shown in the first three rows of Table 4, DeepSeek-V3 achieves the best overall performance in terms of correctness and efficiency. It attains the highest compilation success rate (64.81%) and line coverage (71.56%), while also requiring the least execution time (35.76s) and generating the shortest output length (1104.73 tokens). Kimi-K2-Instruct demonstrates competitive results, achieving the highest branch coverage (70.71%) with a compilation success rate close to DeepSeek-V3 (61.11%). In contrast, DeepSeek-V3.2 exhibits the worst performance regarding compilation success (46.91%) and consumes the most resources (70.73s, 2780.24 tokens).

Performance on High Complexity Dataset. The results on the high-complexity dataset, shown in the last three rows of Table 4, highlight significant performance shifts. DeepSeek-V3 maintains its superiority with the highest compilation success rate (58.62%) and branch coverage (51.61%). Notably, Qwen3-Coder outperforms others in line coverage (38.64%). Conversely, Kimi-K2-Instruct suffers a drastic decline, with its compilation success rate dropping to 41.38% and line coverage falling to a mere 18.42%. DeepSeek-V3.2 continues to underperform, recording the lowest compilation success rate (37.93%) and the longest execution time (150.46s) and output length (5522.28 tokens).

The above results reveal that LLMs with stronger general capabilities do not necessarily perform better in unit test generation, as they may generate overly complex code that is prone to errors. Furthermore, the same LLM can exhibit significant performance differences across tasks of varying difficulty levels. Consequently, model selection is highly task-dependent, and it is crucial to evaluate models specifically on the target application before deployment.

4.2.4 Results for RQ4. Table 5 presents the comparison results on the Defects4J dataset. DEHALL demonstrates superior performance across multiple evaluation metrics. In terms of coverage capability, DEHALL achieves 79.54% line coverage and 73.96% branch coverage, significantly outperforming ChatUniTest and EvoSuite. In terms of efficiency, DEHALL also demonstrates remarkable performance with an average test generation cost time of only 38.89s. For the search-based method EvoSuite, the time is cut up to 60s by the official default setting. For ChatUniTest, the interaction of the


```

public static Number createNumber(final String str)
    throws NumberFormatException {
    // ...
    try {
        final Float f = createFloat(str); // BUG: It loses precision

        if (!f.isInfinite()
            (f.floatValue() == 0.0F && !allZeros))) {
            return f;
        }
    } catch (final NumberFormatException nfe) {
        // ignore the bad number
    }
    // ...
    return createBigDecimal(str); // It provides reference for DEHALL
}

```

(a) Focal Method with a BUG.

```

// -- DEHALL: Targets BigDecimal with high-precision input --
@Test public void testCreateNumber_LargeDecimal() {
    String largeNum = "1.23456789012345678901234567890123456789";
    Number result = NumberUtils.createNumber(largeNum);
    assertTrue(result instanceof BigDecimal);
    assertEquals(new BigDecimal(largeNum), result); }

// -- ChatUnitTest: Tests only Float types --
@Test public void testCreateNumber_Float() {
    assertEquals(1.23f, NumberUtils.createNumber("1.23f"));
    assertEquals(1.23f, NumberUtils.createNumber("1.23F")); }

// -- EvoSuite: Minimal random input --
@Test(timeout = 4000) public void test045() throws Throwable {
    Number number0 = NumberUtils.createNumber(".7");
    assertEquals(0.7F, number0); }

```

(b) Test Case Comparison.

Figure 7: Case study of unique defect discovery: (a) Focal bug method with buggy line highlighted in red; (b) Generated test cases from DEHALL, ChatUnitTest, and EvoSuite. Only DEHALL discovered the bug by testing with a large decimal number.

compiler and executor largely increases the overall time cost. This efficiency advantage is particularly significant in continuous testing scenarios where rapid test generation is critical.

Defect Detection Capability. Beyond coverage metrics, DEHALL demonstrates superior bug detection performance in real-world scenarios. We randomly select 20 focal methods from buggy version on projects Math and Lang within Defects4J, following the setup in previous work [34]. And we manually check the runtime logs for generated test cases to verify the results of bug detection. Our experiments show that DEHALL successfully identifies 11 distinct defects compared to 7 defects by ChatUnitTest and 6 defects by EvoSuite. Figure 7 illustrates this representative case. As shown in Figure 7(a), the focal method `createNumber` contains a subtle bug where it fails to properly handle very large decimal numbers. The problematic implementation first creates a `Float` instance by `createFloat`, which inherently loses precision for large decimals, before falling back to `createBigDecimal` when precision issues arise. The test case comparisons in Figure 7(b) reveal fundamental differences between the three approaches. DEHALL generates a targeted test that specifically exercises the precision boundary by inputting a very long decimal number that triggers the bug, verifying that the result should be a `BigDecimal` instance to maintain full precision. ChatUnitTest’s tests focus only on standard `Float` type with simple decimal inputs, failing to test precision-sensitive scenarios. EvoSuite uses random inputs like `".7"` while restricting the expected output type to `Float`, also missing the precision defect.

The above experiments on both coverage and bug detection prove that the generated test cases by DEHALL have high coverage, superior readability, and rich diversity.

5 Discussion & Lessons Learned

DEHALL has been adopted by thousands of developers at Ant Group for unit test generation in their daily development practices. This large-scale deployment has resulted in significant efficiency gains, as reflected by the increase in the developer adoption rate from 57% to 81%. To assess the quality of the generated tests, we collected qualitative feedback from the users. The responses indicate a high level of satisfaction with the tool’s coverage and stability. For instance, one user (P1) noted:

“It effectively improves code coverage. A single incremental generation run satisfies the coverage requirements for the current development iteration.”

Another user (P2) commented on the usability and reliability:

“It is quite good, and the test pass rate is very high.”

These comments suggest that DEHALL successfully meets the coverage goals of development while maintaining high test reliability.

Besides, based on our evaluation and the analysis of specific cases, we summarize the key lessons learned from our experiments. These insights not only validate our design but also provide practical guidelines for deploying LLM-based unit test generation systems.

Lesson 1: General Capability Does Not Dictate Task-Specific Effectiveness. A common assumption is that LLMs with stronger general coding capabilities will naturally perform better in unit test generation. However, our results reveal a counterintuitive trend. For instance, despite DeepSeek-V3.2 possessing superior general capabilities compared to DeepSeek-V3, it significantly underperforms in generating compilable and high-coverage tests. It indicates that developers must select models based on metrics relevant to specific tasks rather than relying solely on general coding leaderboards. A model optimized for general code completion may introduce excessive complexity or hallucinations when strictly constrained by unit testing requirements.

Lesson 2: Performance Variance Across Task Complexity. Our findings highlight that a model’s performance is not static but varies considerably with the difficulty of the tasks. For example, Kimi-K2-Instruct shows competitive performance on the normal complexity dataset but suffers a drastic decline on the high complexity dataset. In a practical engineering setting, it is advisable to flexibly select different models based on the performance to different complexities of the code under test to ensure optimal results.

Lesson 3: Handling Repetition Loops in Generation. During our experiments and analysis, we observe that LLMs occasionally enter a state of “repetition loops,” where they continuously output repetitive code content without termination. This issue renders the final code unusable and severely degrades user experience by causing timeouts or resource exhaustion. To mitigate this, we recommend implementing robust engineering fallback mechanisms.

For instance, system architectures can invoke multiple LLMs simultaneously for a single user request; if the primary model exhibits abnormal behavior, the system can switch to the output from a backup model to ensure reliability.

Limitations. There still remain challenges to complex syntax and changes of framework versions. While LLMs excel at understanding complex logic and branches, they struggle with API compatibility issues specific to framework versions and advanced syntax abstractions. We find that LLMs often conflate syntaxes with hallucinations for different versions across applications. For instance, the method `mockStatic` for mocking static attributes, which is only supported in Mockito 3.4.0+ and differs entirely from the approach required in earlier versions (e.g., using `PowerMock`). Similarly, complex syntaxes like generics are frequently misinterpreted as concrete classes during generation. These failures suggest that accurate unit test generation requires not just code understanding, but also precise awareness of the specific dependency versions and syntactic nuances in the target applications.

Future Directions. Based on the above discussions, we identify several promising directions for future work:

- *Expanding Static Analysis Scope:* We aim to enhance the test case repair module to handle more complex syntax issues and broader scenarios, reducing the burden of manual fixes.
- *Interactive Agent Modes:* We plan to explore a more flexible agent-based interaction mode. This would allow users to provide natural language feedback on the generated test cases (e.g., "add a null check for this parameter"), enabling iterative refinement and improving the test suite quality.
- *Learning from Target Repositories:* We plan to incorporate mechanisms to learn from the target code repository, such as analyzing existing code patterns and testing conventions, to improve the model's contextual understanding and adaptability to specific project environments.

6 Related Work

While unit testing is a broad domain aimed at verifying isolated modules and interconnected components [24], in this section, we focus specifically on automated unit test generation techniques.

6.1 Traditional Unit Test Generation

Over time, a diverse array of test generation strategies [4, 6, 17] has emerged, ranging from stochastic random sampling to logic-centric constraint-solving techniques. Building upon basic randomization, fuzzing techniques [5, 35, 37] have gained prominence for efficiently uncovering security vulnerabilities by exploring program inputs with high coverage. However, fuzzing techniques focus on generating random test cases aimed at covering corner cases, which limits the maintainability and comprehensibility of test cases in industrial settings. While tools relying on symbolic execution [10, 20, 38] offer comprehensive test generation, they frequently suffer from the path explosion problem [2]. To address these scalability issues, search-based software testing (SBST) methodologies [21] such as `EvoSuite` [8] have been adopted. Although these algorithms mitigate certain limitations, they are themselves burdened by vast search spaces and prohibitive computational overheads.

6.2 LLM-based Unit Test Generation

More recently, the software testing domain has seen a surge in the adoption of Large Language Model (LLM) based techniques. For instance, `TestPilot` [28] constructs tests by synthesizing function signatures, implementations, and usage patterns harvested from documentation. `ChatUniTest` [3] employs ChatGPT to produce Java-compliant test cases following a preliminary static parse and analysis phase with validations by compiler and executor. `ChatTester` [36] focuses on refining raw test outputs via ChatGPT, a process that occasionally surpasses the performance of conventional SBST benchmarks. The trend toward hybridization is evident in tools like `TELPA` [33], which augments LLM prompts with refined counter-examples derived from program analysis to target hard-to-reach branches. The aforementioned approaches fall short of leveraging a repair mechanism as fine-grained and practical as `DEHALL`, resulting in suboptimal performance. Driven by preliminary hallucination analysis, `DEHALL` captures more comprehensive code feature contexts and incorporates a static analysis strategy to refine the generated test cases. `TestCTRL` [39] applies CoT prompt and reinforcement learning strategy to generate unit test cases. `InTUT` [23] proposes to adopt test intentions to guide the generation of unit tests. `HITS` [31] improves the testing of complex methods by decomposing them into slices for sequential generation. However, these techniques often incur significant overhead, requiring either additional fine-tuning of the LLM or multiple LLM queries to produce the necessary guidance. `RefTest` [40] exploits inter-procedural relationships to derive test references. Nevertheless, its performance is significantly limited in cold-start scenarios where no prior test cases are available. In parallel to academic advances, LLM-based unit test generation has also been increasingly adopted in industry [1, 9, 12, 25], where developers use LLMs to draft unit tests for efficiency improvement. These industry solutions are often tailored to specific scenarios, making them difficult to generalize across different industrial settings.

7 Conclusion

In this paper, we present `DEHALL`, an automated unit test generation tool that addresses hallucination challenges in LLM-based unit test generation. Evaluation on Ant Group's datasets shows 71.56% line coverage and 67.18% branch coverage, significantly outperforming vanilla LLM approaches. `DEHALL` also maintains superior performance on the public `Defects4J` benchmark, outperforming both traditional search-based methods (`EvoSuite`) and recent LLM-based approaches (`ChatUniTest`). The tool has been successfully deployed at Ant Group with 81% developer adoption rate, demonstrating a strong practical impact. Beyond quantitative improvements, `DEHALL` provides a practical perspective on deploying LLM-based unit test generation in industrial settings, offering valuable insights for organizations exploring similar applications.

Acknowledgments

We thank the anonymous FSE reviewers for their valuable feedback. We are also grateful to Fan Shao, Xiangjie Bu, Siyu Chen, Ning Wang, and Chang Wei for their helpful contributions. The work is supported by the NSFC Project (Grant No.62502077) and the Fundamental Research Funds for the Central Universities (ZYGX2024XJ036).

References

- [1] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 185–196.
- [2] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A survey of symbolic execution techniques. *Comput. Surveys* 51, 3 (2018), 1–39.
- [3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.
- [4] Lori A. Clarke. 1976. A system to generate test data and symbolically execute programs. *IEEE Transactions on software engineering* 3 (1976), 215–222.
- [5] Peng Di, Bingchang Liu, and Yiyi Gao. 2024. MicroFuzz: An Efficient Fuzzing Framework for Microservices. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 216–227.
- [6] Joe W Duran and Simeon C Ntafos. 2009. An evaluation of random testing. *IEEE transactions on Software Engineering* 4 (2009), 438–444.
- [7] Christof Ebert, James Cain, Giuliano Antoniol, Steve Counsell, and Phillip Laplante. 2016. Cyclomatic complexity. *IEEE software* 33, 6 (2016), 27–29.
- [8] Gordon Fraser and Andrea Arcuri. 2011. Evolutionary generation of whole test suites. In *Proceedings of the 11th International Conference on Quality Software*. 31–40.
- [9] Gwihwan Go, Chijin Zhou, Quan Zhang, Yu Jiang, and Zhao Wei. 2025. LSPAI: An IDE Plugin for LLM-Powered Multi-Language Unit Test Generation with Language Server Protocol. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 144–149.
- [10] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*. 213–223.
- [11] T Hariprasad, G Vidhyagaran, K Seenu, and Chandrasegar Thirumalai. 2017. Software complexity analysis using halstead metrics. In *Proceedings of the 2017 international conference on trends in electronics and informatics*. 1109–1113.
- [12] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-guided llm-based test generation at meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 180–191.
- [13] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [14] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).
- [15] JUnit Team. 1997. *JUnit*. Retrieved 2026-1 from <https://junit.org>.
- [16] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [17] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [18] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [19] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556* (2025).
- [20] Lei Ma, Cyrille Artho, Cheng Zhang, Hiroyuki Sato, Johannes Gmeiner, and Rudolf Ramlar. 2015. Grt: Program-analysis-guided random testing (t). In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering*. 212–223.
- [21] Phil McMinn. 2011. Search-based software testing: Past, present and future. In *Proceedings of the IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. 153–163.
- [22] Mockito Team. 2008. *Mockito*. Retrieved 2026-1 from <https://site.mockito.org>.
- [23] Zifan Nan, Zhaoqiang Guo, Kui Liu, and Xin Xia. 2025. Test intention guided llm-based unit test generation. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*. 779–779.
- [24] Michael Olan. 2003. Unit testing: test early, test often. *Journal of Computing Sciences in Colleges* 19, 2 (2003), 319–328.
- [25] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2025. Aster: Natural and multi-language unit test generation with llms. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice*. 413–424.
- [26] PowerMock Team. 2008. *PowerMock*. Retrieved 2026-1 from <https://github.com/powermock/powermock>.
- [27] Armin Ronacher. 2008. Jinja2 documentation. *Welcome to Jinja2—Jinja2 Documentation (2.8-dev)* (2008).
- [28] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.
- [29] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).
- [30] TestNG Team. 2004. *TestNG*. Retrieved 2026-1 from <https://testng.org>.
- [31] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. Hits: High-coverage llm-based unit test generation via method slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268.
- [32] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [33] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. 2024. Advancing code coverage: Incorporating program analysis with large language models. *ACM Transactions on Software Engineering and Methodology* (2024).
- [34] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1607–1619.
- [35] Jiazhao Yu, Yanlun Tu, Zhanlei Zhang, Tiehua Zhang, Cheng Xu, Weigang Wu, Hong Jin Kang, and Xi Zheng. 2025. Grey-Box Fuzzing in Constrained Ultra-Large Systems: Lessons for SE Community. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 204–214.
- [36] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.
- [37] Joobeom Yun, Fayozbek Rustamov, Juhwan Kim, and Youngjoo Shin. 2022. Fuzzing of embedded systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–33.
- [38] Chengyu Zhang, Yichen Yan, Hanru Zhou, Yinbo Yao, Ke Wu, Ting Su, Weikai Miao, and Geguang Pu. 2018. Smartunit: Empirical evaluations for automated unit testing of embedded software in industry. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*. 296–305.
- [39] Junwei Zhang, Xing Hu, Xin Xia, Shing-Chi Cheung, and Shanping Li. 2025. Automated unit test generation via chain of thought prompt and reinforcement learning from coverage feedback. *ACM Transactions on Software Engineering and Methodology* (2025).
- [40] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2025. Reference-Based Retrieval-Augmented Unit Test Generation. *ACM Transactions on Software Engineering and Methodology* (2025).